

REPASO AVANZADO – UNIDADES 1 a 7

Ejercicio 1: Validación y flujo

Desarrollar un programa que permita ingresar datos de estudiantes (nombre, edad, nota).

Requisitos:

- Validar:
 - Edad > 0
 - Nota entre 0 y 10
- Si hay error:
 - Mostrar mensaje y volver a pedir el dato (no cortar el programa)
- Permitir ingresar múltiples estudiantes hasta que el usuario decida salir
- Mostrar al final:
 - Promedio de notas
 - Cantidad de aprobados (nota $\geq$ 6)

Restricción: separar lógica en funciones.

Ejercicio 2: Encapsulamiento real

Diseñar una clase CuentaBancaria.

Requisitos:

- Atributos privados:
 - titular
 - saldo
- Métodos:
 - depositar(double monto)
 - retirar(double monto)
- Validaciones:
 - No permitir saldo negativo
 - No permitir depósitos negativos

No se puede modificar el saldo directamente desde afuera.

Ejercicio 3: Diseño con constructores y relaciones

Modelar un sistema de pedidos.

Clases:

- Producto
- Pedido

Requisitos:

- Un pedido tiene múltiples productos
- Cada producto tiene:
 - nombre
 - precio
- El pedido debe:
 - calcular total automáticamente
 - inicializarse con al menos un producto

Usar constructores sobrecargados.

Ejercicio 4: UML → Código

Dado el siguiente escenario (NO dibujado):

- Un Autor puede tener varios Libros
- Un Libro pertenece a un solo autor
- Una Biblioteca contiene libros, pero los libros pueden existir fuera de ella

Consigna:

1. Dibujar el diagrama UML
2. Implementar las clases en Java
3. Justificar:
 - Qué relaciones usaste (asociación, agregación o composición)
 - Por qué

Ejercicio 5: Colecciones + lógica

Desarrollar un sistema que gestione una lista de alumnos.

Requisitos:

- Usar ArrayList
- Permitir:
 - Agregar alumnos
 - Buscar alumno por nombre
 - Mostrar alumnos con nota mayor al promedio
- Calcular promedio general

Restricción: usar for-each para recorridos.

Ejercicio 6: Enumeraciones + decisiones

Crear una enum EstadoPedido con:

- PENDIENTE
- ENVIADO
- ENTREGADO
- CANCELADO

Consigna:

- Asociar el estado a un Pedido
- Implementar lógica que:
 - No permita pasar de ENTREGADO a otro estado
 - No permita enviar si está cancelado

El control debe estar dentro del modelo, no en el main.

Ejercicio 7: Herencia + sobrescritura

Crear una jerarquía de empleados.

Clases:

- Empleado (base)
- EmpleadoTiempoCompleto
- EmpleadoPorHora

Requisitos:

- Método calcularSalario()
- Cada tipo lo implementa diferente

- Usar sobrescritura

Ejercicio 8: Polimorfismo real

A partir del ejercicio anterior:

Consigna:

- Crear una lista de empleados (mezclados)
- Recorrerla y mostrar el salario de cada uno

No usar instanceof

Ejercicio 9: Abstract + diseño

Crear una clase abstracta Vehiculo.

Requisitos:

- Método abstracto mover()
- Subclases:
 - Auto
 - Bicicleta
- Cada una implementa su comportamiento

No se puede instanciar Vehiculo.

Evitar duplicación de lógica.

Ejercicio Integrador

Desarrollar un sistema de gestión de cursos.

Requisitos:

- Clase base: Persona
- Clases derivadas:
 - Alumno
 - Profesor
- Clase Curso:
 - Tiene lista de alumnos
 - Tiene un profesor

Condiciones:

- Usar:
 - Herencia
 - Polimorfismo
 - Colecciones
- Permitir:
 - Inscribir alumnos
 - Mostrar información completa del curso

Restricciones:

- No usar atributos públicos
- No lógica en main (solo orquestación)